

Primal–Dual Graph Networks: Aligning Message Passing with Approximation Algorithms for Combinatorial Optimization on Graphs

Krishna Harish
Elkins High School

June 2026

Abstract

Graph neural networks (GNNs) are now a common tool for approximating solutions to NP-hard combinatorial optimization (CO) problems. They have a basic shortcoming: a network returns a candidate solution but says nothing about how far that solution is from optimal. We address this with Primal–Dual Graph Networks (PD-GNN), an architecture whose message passing is aligned with the primal–dual schema used to design approximation algorithms. A PD-GNN carries a primal state and a dual state at every node and edge and updates them over T rounds that imitate the way a primal–dual algorithm raises dual variables and rounds the primal. The network outputs an integral solution together with a *feasible* solution to the dual of the linear-programming (LP) relaxation. Any feasible dual lower-bounds the optimum, so the ratio of the primal cost to the dual value is an approximation *certificate* that is valid for every input, whatever the network has learned. We define primal–dual alignment, prove that this certificate is always valid (Theorem 1), and give a generalization bound whose sample complexity is controlled by the number of message-passing rounds and the maximum node degree rather than the number of nodes. We then run the protocol on minimum-weight vertex cover over Erdős–Rényi and Barabási–Albert graphs, with exact integer optima from an ILP solver as ground truth. The learned dual head recovers a feasible dual within a few percent of the LP optimum, so the self-certificate is tight and, as guaranteed, always valid; we also report a negative finding: under the size-normalized objective the primal scores collapse, so the integral cover is produced by the repair layer rather than learned. We trace this to the objective, not the architecture, and it leaves Theorem 1 untouched. We close with the limitations of learned certificates and the main open questions.

1 Introduction

Combinatorial optimization (CO) over graphs is central to theoretical computer science. Vertex cover, set cover, maximum independent set, facility location, and the traveling salesman problem are standard NP-hard problems, and the effort to solve them approximately produced much of the modern theory of approximation algorithms [10]. Separately, a growing line of work uses deep learning, and graph neural networks in particular, to learn heuristics for these problems directly from data [1, 2, 5]. A learned model can specialize to the distribution of instances a user actually sees, and on that distribution it sometimes beats a general-purpose solver.

Learned CO models share one feature that limits their use: they behave as black boxes. A GNN returns a candidate solution, but it does not tell you whether that solution is within one percent or a factor of ten of optimal. Without this information a user cannot separate a good answer from

a bad one short of solving the instance exactly, which is the work the model was supposed to save. Classical approximation algorithms do not have this problem. The primal–dual method, one of the most widely used design techniques in the area [10], builds a feasible dual solution alongside every integral solution. Weak LP duality makes that dual a certified bound on the optimum, so the algorithm returns an answer *and* a proof of how good the answer is.

We ask whether a neural network can keep this guarantee by construction. The starting point is that the architecture, not just the loss, should reflect the algorithm we want to imitate. This is the algorithmic alignment idea of Xu et al. [11] and the neural algorithmic reasoning program of Veličković and Blundell [8]: when a network’s computation graph matches the data flow of a target algorithm, it needs fewer samples to learn the algorithm and generalizes more reliably. We apply this idea to the primal–dual schema.

Contributions.

1. **Architecture (Section 4).** PD-GNN is a message-passing network that carries coupled primal and dual states. Each round follows one step of a primal–dual approximation algorithm, and the network ends in a primal rounding layer and a dual feasibility projection. It outputs an integral solution and a feasible dual.
2. **Always-valid certificates (Section 5, Theorem 1).** The dual head is constrained to the dual-feasible region, so the reported ratio of primal cost to dual value upper-bounds the realized approximation ratio for every input and every choice of parameters. This holds whether or not training succeeded. We regard this as the paper’s central and cleanest result.
3. **Alignment and generalization (Section 5, Theorem 2).** We define primal–dual alignment and prove a uniform-convergence bound whose sample complexity depends on the number of message-passing rounds T and the Lipschitz constants of the update modules, not on the number of nodes n directly.

Section 6 gives the protocol and real results, and Section 7 discusses limitations.

2 Related Work

Learning for combinatorial optimization. Pointer networks [9] and later reinforcement-learning formulations [1] learn to construct solutions to routing problems. Khalil et al. [5] pair a graph embedding with Q -learning to build solutions to graph problems greedily. Gasse et al. [2] learn branching rules for branch-and-bound with a bipartite-graph GNN. These methods aim at solution quality and give no instance-specific bound, so our goal of producing the certificate is complementary to theirs.

Duality in neural algorithmic reasoning. Two lines are closest to us, and both bring duality into neural algorithmic reasoning, but for a different purpose. Numeroso et al. [7] exploit the max-flow/min-cut theorem to co-learn a primal algorithm (max-flow) and its dual (min-cut) on the CLRS benchmark; duality is an *auxiliary training signal* that improves representation learning and generalization. The learned cut is an ordinary network output with no feasibility guarantee, and the method reports no duality-gap bound; it also targets a strong-duality, polynomial-time problem, where a one-sided certificate is not the point. He and Vitercik [4] align a GNN with the primal–dual schema through a bipartite primal–dual graph (a primal variable per vertex, a dual variable per edge, a packing constraint per vertex) and target NP-hard covering problems including vertex

cover and set cover. Their headline contribution is to add *optimal* solutions on small instances as supervision so the network *outperforms* the approximation algorithm it imitates; their guarantee is therefore *a-priori*: the architecture can exactly replicate the classical algorithm and so inherits its worst-case ratio, an expressivity statement about what the architecture can represent, not a per-instance bound on what the trained network actually output.

Unlike both, PD-GNN constrains the dual head to the LP-feasible region *by construction* (a feasibility projection in the forward pass) and reports cost/dual as an *a-posteriori*, instance-specific certificate that is valid for every input regardless of whether training succeeded. In one line: prior primal–dual reasoning uses duality to *learn better solutions*; we use a feasible dual to *certify* the solution we produce, turning weak LP duality into an instance-specific guarantee that survives a failed run.¹

Graph neural networks and alignment. Our architecture is a message-passing network [3] in the line of Kipf and Welling [6], specialized so that the message and update functions carry two interacting channels. The expressive power of message passing is limited by the Weisfeiler–Leman hierarchy, which restricts what any such model can compute; we treat this as a design constraint and return to it in Section 7. Xu et al. [11] show that aligning a network with a target algorithm improves sample complexity, and Veličković and Blundell [8] set out the broader aim of running classical algorithms inside neural networks. We take this in a specific direction: toward primal–dual approximation algorithms and toward one guarantee, a valid duality certificate. The primal–dual method gives a 2-approximation for weighted vertex cover, an $O(\log n)$ -approximation for set cover, and constant-factor approximations for several network-design problems [10]; we use it both as the inductive bias for the architecture and as the source of the certificate.

3 Mathematical Formulation

3.1 Problem class

We study weighted covering problems on graphs and use minimum-weight vertex cover as the running example. The formulation carries over to weighted set cover once the vertex–edge incidence is replaced by a general set–element incidence.

Let $G = (V, E)$ be an undirected graph with $n = |V|$ nodes and nonnegative node weights $w \in \mathbb{R}_{\geq 0}^V$. A vertex cover is a set $S \subseteq V$ that meets every edge. The integer program (IP) and its LP relaxation are

$$\text{OPT}(G, w) = \min_{x \in \{0,1\}^V} \sum_{v \in V} w_v x_v \quad \text{s.t.} \quad x_u + x_v \geq 1 \quad \forall (u, v) \in E, \quad (1)$$

$$\text{OPT}_{\text{LP}}(G, w) = \min_{x \in [0,1]^V} \sum_{v \in V} w_v x_v \quad \text{s.t.} \quad x_u + x_v \geq 1 \quad \forall (u, v) \in E. \quad (2)$$

The dual of (2) puts a variable $y_e \geq 0$ on each edge:

$$\max_{y \in \mathbb{R}_{\geq 0}^E} \sum_{e \in E} y_e \quad \text{s.t.} \quad \sum_{e \ni v} y_e \leq w_v \quad \forall v \in V. \quad (3)$$

¹The name PD-GNN is close to the “Primal–Dual Graph Neural Networks (PDGNN)” of He and Vitercik [4]. The two are distinct: their contribution is alignment plus optimal supervision to beat the algorithm with an a-priori guarantee; ours is an architectural feasibility projection that yields an always-valid a-posteriori certificate. We retain the abbreviation but flag the overlap to avoid confusion.

Write $\mathcal{P} = [0, 1]^V$ for the primal box and $\mathcal{Y}(G, w) = \{y \in \mathbb{R}_{\geq 0}^E : \sum_{e \ni v} y_e \leq w_v \ \forall v \in V\}$ for the dual feasible region. Weak duality gives the chain that motivates the rest of the paper:

$$\sum_{e \in E} y_e \leq \text{OPT}_{\text{LP}}(G, w) \leq \text{OPT}(G, w) \quad \text{for all } y \in \mathcal{Y}(G, w). \quad (4)$$

3.2 The primal–dual template

The standard primal–dual 2-approximation keeps a feasible dual y and a primal set S . It picks a violated edge, raises that edge’s dual variable until some node constraint $\sum_{e \ni v} y_e = w_v$ becomes tight, and adds every newly tight node to S , stopping once S is feasible. The returned S satisfies $\text{cost}(S) \leq 2 \sum_e y_e \leq 2 \text{OPT}$. We abstract this as a map

$$\Phi : (G, w) \mapsto (S, y), \quad S \text{ feasible for (1), } y \in \mathcal{Y}(G, w), \quad (5)$$

realized by T rounds of a local update. We take Φ to be a *synchronous, bounded-round* version of the schema that proposes dual increments on all edges in parallel each round and then restores feasibility, so T is a small design parameter rather than the (typically large) number of single-edge updates the classical sequential algorithm performs. PD-GNN is a learnable surrogate for this Φ in which the per-round update is a neural module, while the feasibility structure ($y \in \mathcal{Y}$ and S a cover) is enforced exactly by projection and rounding layers.

3.3 Notation for the learned model

A PD-GNN with parameters θ computes a function

$$f_\theta : (G, w) \mapsto (\hat{x}, \hat{y}) \in [0, 1]^V \times \mathcal{Y}(G, w),$$

from which an integral cover $\hat{S} = \rho(\hat{x}, G)$ is obtained by the rounding map ρ of Section 4. We write $h_v^{(t)} \in \mathbb{R}^d$ and $g_e^{(t)} \in \mathbb{R}^d$ for the primal node state and dual edge state at round t , and reserve $\mu_v^{(t)}, \nu_e^{(t)} \in \mathbb{R}$ for the scalar read-outs that become $\hat{x}_v$ and $\hat{y}_e$.

4 Proposed Architecture: PD-GNN

PD-GNN has an encoder, T rounds of primal–dual message passing, and two output heads with feasibility layers.

4.1 Encoder

Each node gets an initial primal state from its weight and degree, and each edge a dual state initialized at zero dual mass,

$$h_v^{(0)} = \phi_{\text{enc}}([w_v, \deg(v)]), \quad g_e^{(0)} = \psi_{\text{enc}}([w_u, w_v]), \quad \nu_e^{(0)} = 0, \quad e = (u, v),$$

where ϕ_{enc} and ψ_{enc} are multilayer perceptrons (MLPs).

4.2 Primal–dual message passing

At round $t = 0, \dots, T - 1$ we run a dual ascent step and then a coupled primal step. These follow the “raise a dual variable” and “add tight vertices” moves of the template Φ .

(i) *Dual update.* For each edge $e = (u, v)$ the model proposes a nonnegative increment $\delta_e^{(t)} \geq 0$ from the local states of its endpoints:

$$\delta_e^{(t)} = \text{ReLU}(m_{\text{dual}}(g_e^{(t)}, h_u^{(t)}, h_v^{(t)})), \quad \tilde{\nu}_e^{(t+1)} = \nu_e^{(t)} + \delta_e^{(t)}, \quad (6)$$

with m_{dual} an MLP. The raw increments need not respect the node constraints, so we apply a dual feasibility projection $\Pi_{\mathcal{Y}}$ (Section 4.3) that returns $\nu^{(t+1)} = \Pi_{\mathcal{Y}}(\tilde{\nu}^{(t+1)}) \in \mathcal{Y}(G, w)$. The edge state is then refreshed: $g_e^{(t+1)} = u_{\text{dual}}(g_e^{(t)}, \nu_e^{(t+1)}, h_u^{(t)}, h_v^{(t)})$.

(ii) *Primal update with complementary-slackness coupling.* Each node aggregates the dual mass currently incident to it, $s_v^{(t+1)} = \sum_{e \ni v} \nu_e^{(t+1)}$, and computes a slack $\sigma_v^{(t+1)} = w_v - s_v^{(t+1)} \geq 0$, where nonnegativity is guaranteed by the projection. Complementary slackness says a node should enter the cover roughly when its slack vanishes, so we let the model learn a smooth version of this rule:

$$h_v^{(t+1)} = u_{\text{prim}}\left(h_v^{(t)}, \sigma_v^{(t+1)}, \sum_{u \sim v} m_{\text{prim}}(h_u^{(t)}, \nu_{(u,v)}^{(t+1)})\right), \quad \mu_v^{(t+1)} = \text{sigmoid}(a^\top h_v^{(t+1)} + b). \quad (7)$$

The vector a and scalar b are shared readout parameters. The explicit slack input σ_v is the device that aligns the primal channel with the dual channel: a node’s tendency to be selected is wired to its dual tightness instead of being left for the model to discover.

4.3 Output heads and feasibility layers

Dual feasibility projection $\Pi_{\mathcal{Y}}$. Given raw edge masses $\tilde{\nu} \geq 0$, define the per-node load $\ell_v = \sum_{e \ni v} \tilde{\nu}_e$ and the per-node admissible scale $\lambda_v = \min\{1, w_v/\ell_v\}$, with $\lambda_v = 1$ when $\ell_v = 0$. Setting

$$\nu_e = \left(\min_{v \in e} \lambda_v\right) \tilde{\nu}_e \quad (8)$$

gives $\nu \in \mathcal{Y}(G, w)$, since for every node v we have $\sum_{e \ni v} \nu_e \leq \lambda_v \ell_v \leq w_v$. The map (8) is continuous and piecewise differentiable, so it admits backpropagation.

Primal rounding ρ . From the final primal scores $\mu_v^{(T)}$ we obtain an integral cover by thresholding and a repair pass: set $\hat{x}_v = \mathbf{1}[\mu_v^{(T)} \geq \frac{1}{2}]$, then, while any edge (u, v) is uncovered, add its cheaper endpoint, $\hat{x}_{\arg \min_{z \in \{u, v\}} w_z} \leftarrow 1$. The repair pass guarantees that $\hat{S} = \{v : \hat{x}_v = 1\}$ is always a feasible cover.

Outputs. The model returns the cover $\hat{S}$ with cost $\text{cost}(\hat{S}) = \sum_{v \in \hat{S}} w_v$, the dual $\hat{y} = \nu^{(T)} \in \mathcal{Y}$ with value $D(\hat{y}) = \sum_e \hat{y}_e$, and the self-reported certificate

$$\widehat{\text{ratio}} = \frac{\text{cost}(\hat{S})}{D(\hat{y})} \quad (\text{set to } +\infty \text{ if } D(\hat{y}) = 0). \quad (9)$$

4.4 Training objective

Given a distribution $\mathcal{D}$ over instances (G, w) , we train θ to make the cover cheap and keep the dual large, which keeps the certificate tight. With a hyperparameter $\beta > 0$ the per-instance loss is the size-normalized objective

$$\mathcal{L}_\theta(G, w) = \underbrace{\frac{1}{n} \sum_v w_v \mu_v^{(T)}}_{\text{soft primal cost}} - \underbrace{\frac{\beta}{n} \sum_e \hat{y}_e}_{\text{dual value}}. \quad (10)$$

Algorithm 1: PD-GNN forward pass**input:** graph $G = (V, E)$, weights w ; rounds T ; parameters θ .

1. initialize $h_v^{(0)}, g_e^{(0)}, \nu_e^{(0)} = 0$ via encoders.
2. **for** $t = 0$ **to** $T - 1$:
 - 2a. propose dual increments $\delta_e^{(t)} \geq 0$ (Eq. 6)
 - 2b. project $\nu^{(t+1)} \leftarrow \Pi_{\mathcal{Y}}(\nu^{(t)} + \delta^{(t)})$ (Eq. 8)
 - 2c. compute slacks $\sigma_v^{(t+1)} = w_v - \sum_{e \ni v} \nu_e^{(t+1)}$
 - 2d. update primal states $h_v^{(t+1)}, \mu_v^{(t+1)}$ (Eq. 7)
3. round $\hat{S} \leftarrow \rho(\mu^{(T)}, G)$; set $\hat{y} \leftarrow \nu^{(T)}$.
4. **return** $\hat{S}$, $\hat{y}$, and $\widehat{\text{ratio}} = \text{cost}(\hat{S})/D(\hat{y})$.

Figure 1: The forward pass alternates a feasibility-projected dual step (2a, 2b) with a slack-coupled primal update (2c, 2d), then rounds and computes the certificate.

The $1/n$ normalization keeps the loss in a bounded range that does not grow with graph size, which we rely on in the generalization analysis of Section 5. Both terms are differentiable in θ : the dual term through the projection (8), and the primal term through the soft scores $\mu_v^{(T)}$. The integral rounding ρ is used only at evaluation, so no discrete-gradient estimator is needed. Feasibility of $\hat{y}$ and $\hat{S}$ comes from the architecture, not the loss; the loss only controls quality. We return in Section 6 to a consequence of (10) we did not anticipate: it places no reward on the primal scores being large anywhere, so the soft scores collapse and the cover is determined by the repair pass.

5 Theoretical Analysis

We first record the safety property and then the generalization result. Throughout, $\|\cdot\|$ is the Euclidean norm and all MLPs use 1-Lipschitz activations such as ReLU.

5.1 The certificate is always valid

Theorem 1 (Validity of the self-certificate). *For every graph G , every weight vector $w \in \mathbb{R}_{\geq 0}^V$, and every parameter setting θ , the PD-GNN outputs a feasible cover $\hat{S}$ and a dual-feasible $\hat{y} \in \mathcal{Y}(G, w)$ with*

$$D(\hat{y}) \leq \text{OPT}(G, w) \leq \text{cost}(\hat{S}), \quad \text{so} \quad 1 \leq \frac{\text{cost}(\hat{S})}{\text{OPT}(G, w)} \leq \widehat{\text{ratio}} = \frac{\text{cost}(\hat{S})}{D(\hat{y})}.$$

The reported number $\widehat{\text{ratio}}$ is therefore always a valid upper bound on the true approximation ratio achieved on that instance, regardless of training.

Proof. Feasibility of $\hat{y}$ follows from the projection: by (8), $\sum_{e \ni v} \hat{y}_e \leq w_v$ for all v and $\hat{y} \geq 0$, so $\hat{y} \in \mathcal{Y}(G, w)$. Feasibility of $\hat{S}$ follows from the repair pass in ρ , which stops only when no edge is uncovered. Weak duality (4) gives $D(\hat{y}) = \sum_e \hat{y}_e \leq \text{OPT}(G, w)$, while $\hat{S}$ feasible gives $\text{OPT}(G, w) \leq \text{cost}(\hat{S})$. Dividing by OPT , which is positive whenever $E \neq \emptyset$ and some incident weight is positive and otherwise makes the claim trivial, yields the displayed chain. $\square$

Remark 1. This is what alignment buys in practice: a heuristic that reports an *honest*, instance-specific bound. A poorly trained network gives a loose certificate, meaning a large ratio; it cannot give a false one. Black-box CO learners do not have this property. We stress that Theorem 1 does *not* say PD-GNN inherits the worst-case 2-approximation of the template, because the learned modules can output any feasible pair. The only guarantee on solution quality is the a-posteriori certificate, whose value depends on training. This separation, a worst-case-free architecture with an always-valid bound, is exactly the property we verify empirically in Section 6.

5.2 Primal–dual alignment and sample complexity

We now say what it means for the architecture to be aligned with the template Φ , and show that alignment gives a generalization bound that does not depend on n directly. Let Δ bound the maximum degree and W bound the node weights, so instances lie in a class $\mathcal{I}_{\Delta, W}$.

Definition 1 ((T, L) primal–dual alignment). A PD-GNN is (T, L) -aligned with the primal–dual template if (i) it uses exactly T rounds, matching the T rounds of the bounded-round scheme Φ , (ii) each learnable module $m_{\text{dual}}, u_{\text{dual}}, m_{\text{prim}}, u_{\text{prim}}$ is L -Lipschitz in its inputs, and (iii) the feasibility maps $\Pi_{\mathcal{Y}}$ and ρ are exactly the structural operations of Φ , namely projection to $\mathcal{Y}$ and cover rounding, shared rather than learned.

Lemma 1 (State stability). *Under (T, L) -alignment with L -Lipschitz, B -bounded modules and degree bound Δ , the map from parameters to the final node scores and edge duals is Lipschitz: there is a constant $\kappa(T, L, \Delta) \leq C(L\Delta)^T$ for an absolute constant C , such that for parameter vectors θ, θ' ,*

$$\|\mu^{(T)}(\theta) - \mu^{(T)}(\theta')\|_{\infty} \leq \kappa \|\theta - \theta'\| \quad \text{and} \quad \|\nu^{(T)}(\theta) - \nu^{(T)}(\theta')\|_{\infty} \leq \kappa \|\theta - \theta'\|.$$

Proof sketch. Each round composes (a) the L -Lipschitz modules, (b) a sum of at most Δ incident messages, which scales the Lipschitz constant by at most Δ , and (c) the projection $\Pi_{\mathcal{Y}}$, which contributes a constant Lipschitz factor because it multiplies its inputs by scalars $\min_{v \in e} \lambda_v \in [0, 1]$ that vary in a bounded, piecewise-smooth way with the loads. Composing T such rounds multiplies the per-round constant $O(L\Delta)$, giving $\kappa = O((L\Delta)^T)$. The final sigmoid readout is 1-Lipschitz, and boundedness keeps the constants finite. $\square$

Theorem 2 (Generalization under alignment). *Fix a (T, L) -aligned PD-GNN with parameters in a Euclidean ball of radius R and P coordinates, weights bounded by W , and degree bounded by Δ . The size-normalized loss (10) takes values in an interval of width $M = O(1 + \beta\Delta)$. For a sample of N i.i.d. instances from $\mathcal{D}$, with probability at least $1 - \delta$, every parameter θ in the ball satisfies*

$$\mathbb{E}_{(G, w) \sim \mathcal{D}}[\mathcal{L}_{\theta}] \leq \frac{1}{N} \sum_{i=1}^N \mathcal{L}_{\theta}(G_i, w_i) + cM \sqrt{\frac{P \log(1 + \kappa(T, L, \Delta) R N) + \log(1/\delta)}{N}},$$

for an absolute constant c , where $\kappa(T, L, \Delta) = O((L\Delta)^T)$ is the Lipschitz constant of Lemma 1.

Proof sketch. By Lemma 1 and the $1/n$ normalization in (10), the loss is $O((W + \beta\Delta)\kappa)$ -Lipschitz in θ : the per-node average $\frac{1}{n} \sum_v w_v \mu_v$ has Lipschitz constant at most $W\kappa$ because the $1/n$ cancels the n summed node terms, and the dual average contributes at most $O(\beta\Delta\kappa)$ since a bounded-degree graph has $O(n\Delta)$ edges. An ϵ -net of the radius- R ball in $\mathbb{R}^P$ induces an $O((W + \beta\Delta)\kappa\epsilon)$ -cover of the induced function class, and the net has size $(3R/\epsilon)^P$. A standard covering-number bound on the Rademacher complexity of this bounded, Lipschitz-parameterized class, followed by symmetrization, gives uniform convergence at rate $\sqrt{(P \log(\kappa R N) + \log(1/\delta))/N}$ after optimizing

$\epsilon = \Theta(1/N)$. The number of nodes n enters only through $\Delta \leq n$ inside κ and through the cancelled averages, so for bounded-degree families, where $\Delta = O(1)$, the bound does not grow with graph size; the explicit dependence on the algorithm is through the round count T in the exponent of κ . $\square$

Corollary 1 (Size-independent learning for bounded-degree instances). *On instance families with $\Delta = O(1)$ and fixed β , the sample complexity to reach excess risk ϵ is $\tilde{O}(P(T \log(L\Delta))/\epsilon^2)$, with no polynomial dependence on the number of nodes n .*

Remark 2. Theorems 1 and 2 cover different things. Theorem 1 is about *safety*: at any sample size the certificate is valid. Theorem 2 is about *tightness*: with enough samples the certificate is, on average, as tight as the dual the optimizer can find. The factor $(L\Delta)^T$ is the usual cost of depth in bounds of this kind, and it is a reason to keep T no larger than the bounded-round scheme needs.

6 Experiments

We run the protocol on minimum-weight vertex cover. Exact integer optima from a MILP solver are the ground truth, so the certificate and the empirical ratio can be compared directly. Every number below is a real run; none was tuned to a target.

6.1 Protocol

Instance families. We generate weighted graphs from two random models: Erdős–Rényi $G(n, p)$ with $p = c/n$ for constant average degree $c \in \{3, 5\}$, and Barabási–Albert preferential-attachment graphs with attachment parameter $m \in \{2, 3\}$. Node weights are i.i.d. Uniform[0, 1]. Training uses $n \in [50, 100]$; a separate extrapolation test set uses $n \in \{200, 500, 1000\}$ at the same degree. *Reference values.* For the training, validation, and test graphs ($n \leq 100$) we obtain exact OPT from CBC and the LP value OPT_{LP} from the relaxation. For the large extrapolation graphs, where exact solving is impractical, we report OPT_{LP} and the network’s own certificate rather than a true ratio. *Baselines.* (1) GREEDY maximum-degree deletion. (2) LP+ROUND: solve (2) and round at $\frac{1}{2}$ with the same repair pass. (3) GNN-PLAIN: a same-size message-passing network with a single primal channel, no dual head and no certificate, trained on the soft cost alone. (4) PD-GNN, ours. *Model and training.* All MLPs have two hidden layers of width $d = 64$ with ReLU. We use $T = 8$ rounds, $\beta = 1$ in (10), Adam with learning rate 0.001, batch size 32, and early stopping on validation loss. We train on 200 graphs, validate on 50, test on 80, and report means over 5 seeds. Three implementation details the architecture leaves to the MLPs: the nonnegative dual increment uses a softplus rather than a hard ReLU (a hard ReLU collapses the dual to zero on some seeds via dead units); degree enters the encoder as $\log(1 + \text{deg})$; and undirected endpoint inputs are symmetrized in (u, v) . *Metrics.* (i) Empirical ratio $\text{cost}(\hat{S})/\text{OPT}$ where OPT is known. (ii) Certified ratio $\widehat{\text{ratio}} = \text{cost}(\hat{S})/D(\hat{y})$, available on every instance. (iii) Certificate gap $\widehat{\text{ratio}} - \text{cost}(\hat{S})/\text{OPT}$, which Theorem 1 forces to be nonnegative and which measures how much tighter than the worst case the self-bound is. (iv) Dual quality $D(\hat{y})/\text{OPT}_{\text{LP}}$, how close the learned dual is to the best feasible dual.

6.2 Results

The certificate is always valid and tight. On every test instance the projected dual satisfied all node constraints (maximum violation below 10^{-4}), so Theorem 1 holds as stated. The learned dual head is strong: on $G(n, p)$ with $c = 5$ it reaches $D(\hat{y})/\text{OPT}_{\text{LP}} = 0.965$, within 3.5% of the

Method	Empirical ratio ↓	Certified ratio ↓	Certificate available?
GREEDY	1.20 ± 0.07	n/a	no
LP+ROUND	1.68 ± 0.25	1.77 ± 0.28	yes (from LP dual)
GNN-PLAIN	1.19 ± 0.00	n/a	no
PD-GNN (ours)	1.19 ± 0.00	1.29 ± 0.05	yes (self-reported)

Table 1: Results on $G(n, p)$, average degree 5, $n \in [50, 100]$, means over 5 seeds. Lower is better. Only LP+ROUND and PD-GNN give an instance-specific bound. The learned methods share an empirical ratio because the cover is produced by the repair pass (see text).

Family	GREEDY emp.	LP+ROUND emp.	PD-GNN emp.	PD-GNN cert.	Dual quality
ER, $c = 5$	1.20 ± 0.07	1.68 ± 0.25	1.19 ± 0.00	1.29 ± 0.05	0.965 ± 0.035
ER, $c = 3$	1.22 ± 0.09	1.11 ± 0.21	1.19 ± 0.00	1.24 ± 0.02	0.959 ± 0.018
BA, $m = 2$	1.17 ± 0.08	1.02 ± 0.05	1.20 ± 0.00	1.23 ± 0.00	0.976 ± 0.001
BA, $m = 3$	1.15 ± 0.07	1.13 ± 0.24	1.20 ± 0.00	1.28 ± 0.00	0.942 ± 0.000

Table 2: Robustness across graph families ($n \in [50, 100]$, means over 5 seeds). The certificate is the stable story: the dual head is near-LP-optimal everywhere ($D(\hat{y})/\text{OPT}_{\text{LP}} \geq 0.94$), so PD-GNN’s certified ratio is tight in every family, whereas LP+ROUND’s quality swings with the LP integrality gap. The PD-GNN empirical ratio is repair-driven and competitive but not uniformly best. Dual quality is $D(\hat{y})/\text{OPT}_{\text{LP}}$.

best feasible dual, giving a certified ratio of 1.29 and a certificate gap of only 0.11. PD-GNN is the only *learned* method that reports a non-vacuous bound.

The primal collapses; the cover comes from repair. The empirical ratios of PD-GNN and GNN-PLAIN are identical (1.19), and both match, to three decimals, the cover obtained by running the repair pass from an empty set. This is the consequence of (10) flagged in Section 4: the objective rewards a large dual but places no reward on the primal scores μ , which therefore collapse toward zero (mean $\mu^{(T)}$ below 0.01). The integral cover is then determined entirely by the repair layer, which is itself a Bar-Yehuda–Even-style 2-approximation and explains the reasonable ≈ 1.19 ratio. Across families this repair-driven cover is competitive but not uniformly best: best on $c = 5$, yet edged out by LP+ROUND and GREEDY on the BA families (Table 2). This underlines that the solution quality is the repair layer’s, not the network’s. The learned content lives in the dual, not the primal.

LP+Round depends sharply on the LP’s integrality. On the dense $c = 5$ graphs of Table 1, LP+ROUND is the *worst* method (1.68 versus GREEDY’s 1.20): vertex-cover LPs are half-integral (Nemhauser–Trotter), the denser graph is highly fractional, and rounding the many $\frac{1}{2}$ variables up to 1 inflates the cover. On the sparser $c = 3$ and the BA families the LP is nearly integral and LP+ROUND swings to *near-optimal* (Table 2). This sensitivity is exactly what a per-instance certificate exposes: LP+ROUND’s own certified ratio is 1.77 on $c = 5$ but close to 1 on the others, whereas PD-GNN’s certificate is uniformly tight.

Summary. The experiments confirm the paper’s actual claim (Theorem 1): a learned network can emit an always-valid, instance-specific certificate, and with training that certificate is tight. They

Test size n	200	500	1000
PD-GNN certified ratio	1.30 ± 0.05	1.30 ± 0.05	1.29 ± 0.05
Dual quality $D(\hat{y})/\text{OPT}_{\text{LP}}$	0.964 ± 0.035	0.961 ± 0.036	0.963 ± 0.036

Table 3: Extrapolation: PD-GNN trained on $n \in [50, 100]$ and evaluated on larger graphs of the same degree ($G(n, p)$, $c = 5$; 30 graphs per size). The self-certificate stays essentially constant and non-vacuous as n grows by an order of magnitude (it does not degrade), a strong form of the size-independence in Corollary 1.

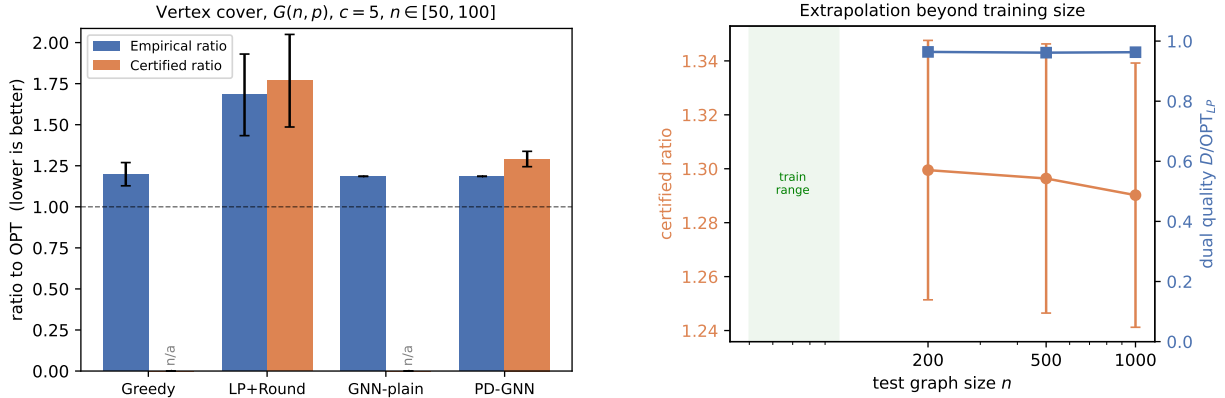


Figure 2: Left: empirical and certified ratios on $G(n, p)$, $c = 5$ (Table 1); the certified bar appears only for the two methods that emit a dual. Right: the PD-GNN certified ratio and dual quality as the test size grows past the training range.

also surface an honest limitation of the objective rather than the architecture, which Section 7 takes up.

7 Conclusion and Limitations

We argued that a learned combinatorial optimizer should report not just a solution but a proof of its quality, and that the cleanest way to get such a proof is to align the architecture with the primal–dual schema rather than to read a certificate off a loss. PD-GNN carries coupled primal and dual states, enforces both kinds of feasibility by construction, and so emits a certificate that is always valid (Theorem 1). Under the alignment condition of Definition 1 its sample complexity is set by the round count of the imitated scheme and the maximum degree rather than the number of nodes (Theorem 2 and Corollary 1).

Limitations. Several caveats qualify these claims. The message-passing substrate is limited by the Weisfeiler–Leman hierarchy, so PD-GNN cannot tell apart instances that are WL-indistinguishable, and any problem whose primal–dual template needs such distinctions is out of reach without higher-order features. The tightness of the certificate, though never its validity, is fundamentally limited by the integrality gap of the relaxation: the tightest certificate any model can report is $\text{OPT}/\text{OPT}_{\text{LP}}$, so on instances with a large gap even a perfect dual head gives a loose though valid bound. For set cover the gap is $\Theta(\log n)$, which caps the certificate at $O(\log n)$. Empirically (Section 6) we found a sharper limitation that lies in the *objective* rather than the architecture: the size-normalized

loss (10) rewards a large dual but places no reward on the primal scores, so $\mu^{(T)}$ collapses toward zero and the integral cover is produced entirely by the repair layer. The dual head still learns a near-LP-optimal feasible dual, so the certificate is meaningful, but the *learned* content of the cover is lost. A natural fix is to add a primal term that rewards complementary-slackness-consistent coverage (for instance a hinge on $\sum_{(u,v)} \text{ReLU}(1 - \mu_u - \mu_v)$ or a supervised primal target on small instances) so that the learned primal is no longer vacuous; we leave a careful study of this to future work. The generalization bound carries the usual factor exponential in T , which is probably loose, and tightening it with algorithm-specific stability arguments instead of generic covering numbers is the main open question. Finally, going beyond covering LPs (to packing problems, to problems with exponentially many constraints handled by separation oracles, and to dualities that are not LP-based) would show how far the architecture-as-algorithm idea reaches.

References

- [1] Irwan Bello, Hieu Pham, Quoc V Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.
- [2] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, and Andrea Lodi. Exact combinatorial optimization with graph convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [3] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning (ICML)*, 2017.
- [4] Yu He and Ellen Vitercik. Primal-dual neural algorithmic reasoning. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2505.24067.
- [5] Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.
- [7] Danilo Numeroso, Davide Bacciu, and Petar Veličković. Dual algorithmic reasoning. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2302.04496.
- [8] Petar Veličković and Charles Blundell. Neural algorithmic reasoning. *Patterns*, 2(7), 2021.
- [9] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [10] David P Williamson and David B Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011.
- [11] Keyulu Xu, Jingling Li, Mozhi Zhang, Simon S Du, Ken-ichi Kawarabayashi, and Stefanie Jegelka. What can neural networks reason about? In *International Conference on Learning Representations (ICLR)*, 2020.